

1C Full Integration App — Technical Spec (T3)

Oct 2, 2026 · @Marshall

Goal and scope

Build a web app that connects to 1C (Бухгалтерия для Узбекистана 3.0) and covers five functions: A) schet-faktura entry, B) analytics, C) error correction, D) auto audit, E) online work. Every task below is sized at 4 hours or less.

Assumptions

- Pilot on your 4 companies: CRYCTAL WATER TREDE MCHJ, TEXMASH IMPORT MCHJ, ООО CRYCTAL WATER TREDE, Техмаш импорт.
- Built product-ready from day one: every table carries `company_id`, so outsourcing clients can be added later without a rewrite.
- All development and testing runs on **copies** of the bases (TEST_CRYSTAL first). A live base is connected only after the acceptance tests pass.
- Solo developer (you), with Claude Code writing most of the code.

Out of scope for v1

- Mobile app (the web app is responsive instead).
- Payroll (Зарплата) and HR documents.
- Modifying the 1C base configuration. Everything on the 1C side lives in an extension (.cfe).

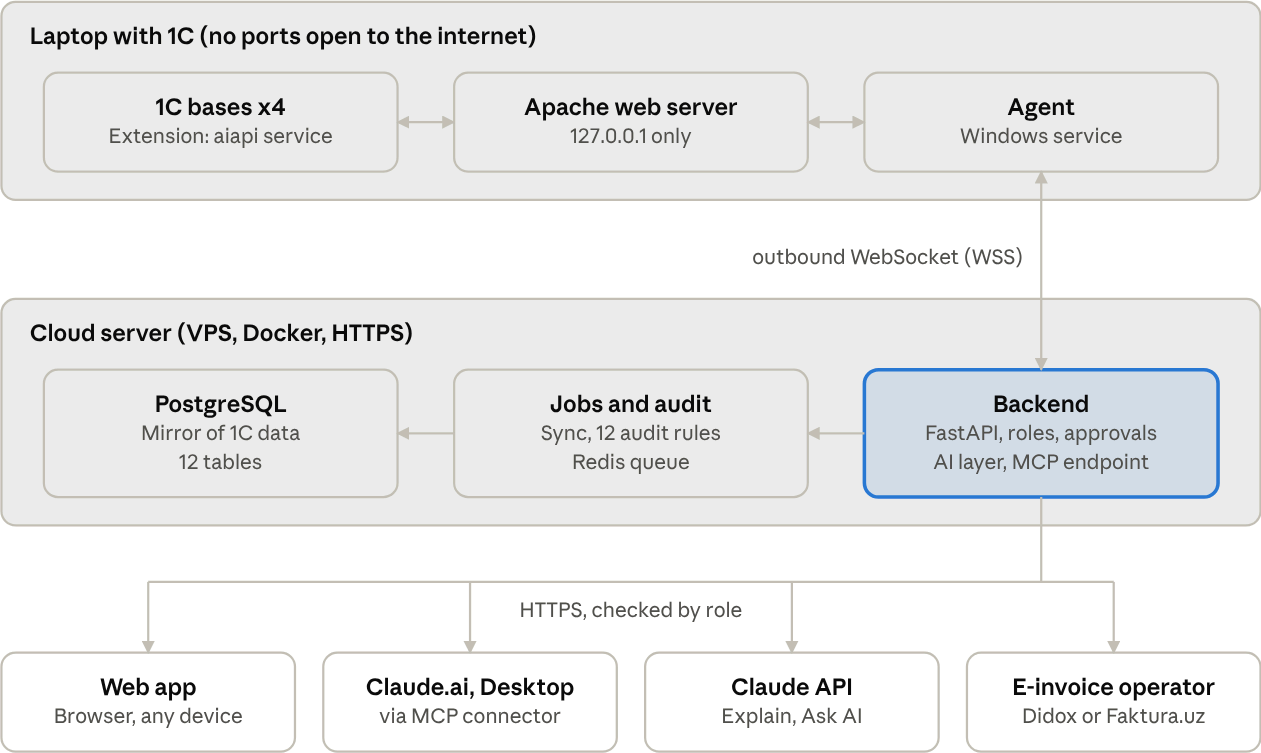
Open questions

- Which e-invoicing operator do the companies use today (Didox, Faktura.uz, Soliq)? This decides Function A's integration.
- File bases or client-server (SQL) bases? File bases are assumed below.

Architecture

The agent is the only part that touches 1C, and it connects outward, so the bases never face the internet.

Only the agent touches 1C; everything else talks to the cloud backend



system architecture · laptop, cloud, users

Claude is wired in twice: the backend calls the Claude API for audit explanations and Ask AI, and its MCP endpoint lets Claude.ai or Claude Desktop query all four companies directly.

Tech stack and repo layout

One repo, five parts, all chosen so Claude Code can generate most of them quickly.

Part	Technology	Why
1C extension	BSL, configuration extension (.cfe), HTTP services	Reads and writes 1C data without changing the base config
Agent	Python 3.12, packaged as a Windows service (.exe via PyInstaller)	Runs next to 1C, calls the extension locally, connects outbound to the backend
Backend	Python, FastAPI, SQLAlchemy, Alembic	Fast to write, async, auto-generated API docs

Part	Technology	Why
Database	PostgreSQL 16	Mirror of 1C data for analytics and audit
Queue	Redis + RQ	Sync jobs, audit runs, outgoing commands to agents
Web app	React + Vite + TypeScript, Tailwind, Recharts	Dashboard and forms
AI	Claude API	Explains audit findings, answers questions about the books
Hosting	One VPS (Docker Compose), Caddy for HTTPS	Cheap, one command to deploy

```

1c-integration/
  extension/      # 1C .cfe source (exported to files)
  agent/         # Windows agent
  backend/       # FastAPI app, models, audit rules
  web/           # React app
  deploy/        # docker-compose.yml, Caddyfile
  docs/          # this spec, API contract

```

1C extension API contract

The extension publishes one HTTP service, `aiapi` , at `/hs/aiapi/v1/` . Only the agent calls it, on `127.0.0.1` , with a Bearer token. All responses are JSON, UTF-8, dates in ISO 8601, amounts as numbers in UZS.

Method	Endpoint	Function	Purpose
GET	<code>/ping</code>	All	Version, base name, INN, 1C platform version
GET	<code>/catalogs/{name}?changed_since=</code>	B, D	Контрагенты, Номенклатура, Договоры, Склады, Организации
GET	<code>/documents/{type}?from=&to=</code>	B, D	Document headers and rows for a period

Method	Endpoint	Function	Purpose
GET	/ledger?from=&to=	B, D	Journal entries (ДТ/КТ, amount, subconto)
GET	/balances?date=&account=	B, D	Account balances and turnovers (OCB)
GET	/changes?since=	E	Objects changed since a timestamp, for incremental sync
POST	/invoices	A	Create a Счет-фактура выданный (unposted draft)
POST	/invoices/{id}/post	A	Post the invoice
POST	/fixes	C	Apply one approved correction (field change, re-post, or reversal)
GET	/fixes/{id}	C	Result of a correction, with before and after values

Rules for every write endpoint

- Requires an `approval_id` issued by the backend. No approval, no write.
- Runs inside one 1C transaction; any error rolls back the whole change.
- Writes a log record to an extension register `ЖурналИзмененийAI` (who, when, object, before, after).
- Refuses to touch a closed period (date before `ДатаЗапретаИзменения`).

Error format: `{"error": "code", "message": "text", "details": {}}` with HTTP 400, 401, 404, 409 (closed period), or 500.

Backend data model

Twelve PostgreSQL tables. Every business table carries `company_id`, and 1C objects keep their 1C reference (`ref_1c`, the UUID) so updates match exactly.

Table	Key columns	Used by
<code>companies</code>	id, name, inn, base_path, agent_id, sync_cursor	All

Table	Key columns	Used by
<code>users</code>	id, email, password_hash, role (owner, accountant, viewer)	E
<code>user_companies</code>	user_id, company_id	E
<code>agents</code>	id, company_id, token_hash, last_seen, version	E
<code>counterparties</code>	company_id, ref_1c, name, inn, contract_refs	A, B, D
<code>items</code>	company_id, ref_1c, name, unit, vat_rate, ikpu_code	A, D
<code>documents</code>	company_id, ref_1c, type, number, date, posted, counterparty_ref, amount, vat, raw_json	B, D
<code>ledger_entries</code>	company_id, document_ref, date, dt_account, kt_account, amount, subconto_json	B, D
<code>invoices</code>	company_id, ref_1c, number, date, buyer_inn, total, vat, status (draft, posted, sent, signed, rejected), operator_id	A
<code>audit_findings</code>	company_id, rule_code, severity, object_ref, amount, message, ai_explanation, status (open, fixed, ignored)	C, D
<code>fixes</code>	id, finding_id, proposed_change_json, approved_by, approved_at, applied_at, result, before_json, after_json	C
<code>event_log</code>	user_id, company_id, action, object_ref, at	All

A) Schet-faktura entry

The user fills one form in the web app; the system creates the invoice in 1C, posts it, and then sends it to the e-invoicing operator.

Flow

1. User picks the company, buyer (search by name or INN), contract, and items. Items autofill unit, price, VAT rate, and IKPU code from `items`.
2. Backend validates: buyer INN is 9 digits, contract exists, every item has an IKPU code, VAT matches the rate, totals add up.

- Backend sends `POST /invoices` through the agent; 1C creates an unposted draft and returns its number.
- User reviews the 1C number and totals, clicks **Post**; `POST /invoices/{id}/post`.
- User clicks **Send to operator**; backend calls the operator API and stores the status (sent, signed, rejected).

Requirements

- Save as draft at any step; nothing reaches 1C until step 3.
- Copy an existing invoice as a template.
- Bulk entry: upload an Excel file with many invoices, preview the validation errors, then create them all.
- Operator integration sits behind one interface (`EinvoiceProvider`), so Didox, Faktura.uz, or Soliq can be swapped in. v1 ships with one provider; until its API is confirmed, step 5 is a stub that only marks the invoice ready to send.

B) Analytics

One dashboard per company plus a combined view of all four, built from the PostgreSQL mirror so 1C is never slowed down.

Widget	Source	Filters
Cash and bank balance (5010 Касса, 5110 Расчётный счёт)	<code>ledger_entries</code>	Company, date
Receivables and payables, top 10 counterparties	<code>ledger_entries</code> on 40xx, 60xx accounts	Company, date, counterparty
Debt aging: 0–30, 31–60, 61–90, 90+ days	<code>documents</code> + <code>ledger_entries</code>	Company, date
Sales and purchases by month	<code>documents</code>	Company, period
VAT: output vs input, VAT payable	<code>invoices</code> , <code>ledger_entries</code>	Company, month
OCB (trial balance)	<code>ledger_entries</code>	Company, period, account
Open audit findings by severity	<code>audit_findings</code>	Company

Requirements

- Every number clicks through to the documents behind it.
- Export any table to Excel.
- **Ask AI** box: a question in Uzbek, Russian, or English ("Why is Kacca negative?"). Backend builds a SQL query over the mirror, runs it read-only, and Claude explains the result.
- Dashboard loads in under 2 seconds for one year of data.

C) Error correction

The system never changes the books on its own. It proposes a fix, a person approves it, and only then does the agent write to 1C.

Flow

1. An audit finding (Function D) has a fix type attached, or the user opens a document and requests a fix.
2. Backend builds the proposed change and shows it side by side: current value, new value, affected entries, and Claude's explanation of why.
3. A user with the `owner` or `accountant` role clicks **Approve**. This creates an `approval_id`.
4. Agent calls `POST /fixes`; 1C applies it in one transaction and returns before and after values.
5. Backend re-syncs the affected objects, re-runs the rule, and closes the finding only if it no longer fires.

Fix types in v1

Fix type	What it changes in 1C
Fill missing field	Sets one empty attribute (INN, contract, IKPU code, VAT rate)
Re-post document	Re-posts a document whose entries don't match its rows
Correct VAT rate	Changes the VAT rate on document rows and re-posts
Reverse duplicate	Marks a duplicate document for deletion and unposts it

Fix type	What it changes in 1C
Merge counterparties	Points documents from a duplicate counterparty to the main one

Requirements

- Undo: every fix stores `before_json`, and a reverse fix can be applied the same way.
- Bulk approve: up to 50 fixes of the same type in one click, each still logged on its own.
- Closed periods are never changed; those findings get the note "correct in the current period".

D) Auto audit

After every sync (and nightly at 02:00), the backend runs a set of rules over the mirror and writes each hit to `audit_findings`. Claude then adds a plain-language explanation and the likely cause.

Code	Rule	Severity	Fix type (C)
CASH-NEG	Cash (5010) balance below zero on any day	Critical	None, investigate
DUP-DOC	Same counterparty, amount, and date on two documents	High	Reverse duplicate
DUP-CP	Two counterparties with the same INN	Medium	Merge counterparties
NO-INN	Counterparty without INN, or INN not 9 or 14 digits	Medium	Fill missing field
NO-IKPU	Sold item without an IKPU code	High	Fill missing field
VAT-RATE	Row VAT differs from the item's VAT rate	High	Correct VAT rate
VAT-MISMATCH	Invoices' VAT total differs from the VAT ledger for the month	Critical	None, investigate
UNPOSTED	Document older than 3 days still unposted	Low	None, reminder

Code	Rule	Severity	Fix type (C)
ENTRY-MISMATCH	Document total differs from the sum of its ledger entries	High	Re-post document
NEG-STOCK	Item quantity below zero at a warehouse	High	None, investigate
OLD-DEBT	Receivable older than 90 days	Medium	None, reminder
NO-CONTRACT	Document without a contract	Low	Fill missing field

Requirements

- Each rule is one Python function with a test; adding a rule takes under an hour.
- A finding ignored by a user stays hidden until the underlying data changes.
- Monthly audit report per company: findings by severity, fixed vs open, exported to PDF.

E) Online work

Users work from any browser, anywhere, while the 1C bases stay on the laptop with no port opened to the internet.

How the agent connects

- The agent opens an outbound WebSocket to `wss://<your-domain>/agent` and keeps it alive (heartbeat every 30 seconds, reconnect with backoff).
- The backend sends commands down that socket (sync, create invoice, apply fix); the agent calls the local extension and returns the result.
- If the laptop is offline, reads come from the mirror (marked with "last synced" time) and writes wait in a queue until the agent reconnects.

Sync

- First sync: full load of catalogs and the current plus previous year of documents and entries.
- Then incremental every 5 minutes via `/changes?since=`, and on demand with a **Sync now** button.

Users and roles

Role	Can do
Owner	Everything, manage users and companies
Accountant	Create invoices, approve fixes, ignore findings
Viewer	Dashboards and reports only

Your brother gets his own login with the role you choose and access only to the companies you assign.

Security requirements

Accounting data for four companies sits behind this app, so these are required for v1, not later.

- HTTPS everywhere (Caddy, automatic certificates).
- Extension HTTP service listens only on `127.0.0.1` ; only the agent's token is accepted.
- Agent token stored hashed on the backend; one token per company, revocable.
- Passwords hashed with Argon2; login sessions expire after 12 hours; optional 2FA (TOTP) for owners.
- Every query filtered by the user's `company_id` list on the backend, never only in the web app.
- Writes to 1C only with an `approval_id` , logged in both the backend and 1C.
- **Ask AI** queries run on a read-only database user, with a 10-second timeout.
- Data sent to Claude: aggregated figures and the finding's details only. Option to replace counterparty names and INNs with tokens before sending.
- Daily PostgreSQL backup, kept 14 days. 1C bases keep their own backups as today.
- Secrets in a `.env` file on the server, never in the repo.

Task breakdown in hours

53 hours in total, split into 13 sessions of about 4 hours. Each session ends with something that works and can be tested. Session 1 starts today.

Session 1 — Foundation (today, 4h). Done when `/ping` returns JSON in the laptop's browser.

- ☐ Create the repo and `docker-compose.yml` with PostgreSQL, Redis, and a FastAPI hello endpoint — 1h
- ☐ Make test copies of all 4 bases — 0.5h
- ☐ Install Apache 2.4 and the 1C web module; publish TEST_CRYSTAL on 127.0.0.1 only — 1h
- ☐ Extension skeleton: `aiapi` HTTP service, token check, `/ping` — 1.5h

Session 2 — Read API (4h). Done when every read endpoint returns real data from TEST_CRYSTAL.

- ☐ `/catalogs/{name}` with `changed_since` — 1.5h
- ☐ `/documents/{type}` with headers and rows — 1.5h
- ☐ `/ledger` and `/balances` — 1h

Session 3 — Agent and first sync (4h). Done when the mirror holds TEST_CRYSTAL's catalogs, documents, and entries.

- ☐ SQLAlchemy models and Alembic migrations for all 12 tables — 1h
- ☐ Agent: WebSocket client, heartbeat, reconnect, command handler — 1.5h
- ☐ Backend: agent endpoint and full-sync job — 1.5h

Session 4 — Incremental sync, users, web shell (4h)

- ☐ `/changes?since=` and the 5-minute incremental sync — 1h
- ☐ Auth: login, Argon2, roles, company access filter — 2h
- ☐ React app: login page, layout, company switcher — 1h

Session 5 — Analytics B (4h)

- ☐ Cash and bank widget, OCB table — 1h
- ☐ Receivables, payables, debt aging — 1.5h
- ☐ Sales and purchases by month, VAT widget — 1h
- ☐ Click-through to documents, Excel export — 0.5h

Session 6 — Auto audit D (4h)

- ☐ Rule engine and the first 6 rules with tests (CASH-NEG, DUP-DOC, DUP-CP, NO-INN, NO-IKPU, VAT-RATE) — 2h
- ☐ The other 6 rules with tests — 1.5h
- ☐ Findings page with filters and ignore — 0.5h

Session 7 — AI layer (5h)

- ☐ Claude explanation for each finding, with name anonymization option — 1h
- ☐ **Ask AI** box: question → read-only SQL → explained answer — 2h
- ☐ Monthly audit report as PDF — 1h
- ☐ MCP endpoint on the backend (`/mcp` , HTTPS, per-user token, read tools for all companies, write tools only through approvals) so Claude.ai and Claude Desktop connect as a custom connector — 1h

Session 8 — Error correction C, core (4h)

- ☐ 1C `/fixes` : transaction, closed-period check, ЖурналИзмененийAI register — 2h
- ☐ Backend: proposal, approval, `approval_id` , apply, re-check — 1h
- ☐ UI: side-by-side before and after, Approve button — 1h

Session 9 — Error correction C, fix types (4h)

- ☐ The 5 fix types in 1C and backend — 2.5h
- ☐ Undo and bulk approve — 1.5h

Session 10 — Schet-faktura A, core (4h)

- ☐ 1C `/invoices` and `/invoices/{id}/post` — 2h
- ☐ Invoice form: buyer search, contract, items with autofill — 2h

Session 11 — Schet-faktura A, complete (4h)

- ☐ Validation rules and copy-as-template — 1h
- ☐ Bulk entry from Excel with preview — 1.5h
- ☐ `EinvoiceProvider` interface and stub provider — 1h
- ☐ Buffer for fixes from sessions 1-10 — 0.5h

Session 12 — Deploy and connect all bases (4h)

- ☐ VPS, domain, Caddy, Docker Compose deploy — 1.5h
- ☐ Agent as a Windows service with an installer — 1.5h
- ☐ Daily backups; publish and connect bases 2-4 (test copies) — 1h

Session 13 — Acceptance testing (4h)

- ☐ Run every test in the next section on all 4 test copies, fix what fails — 4h

Acceptance criteria and test plan

v1 is done when every test below passes on all 4 test copies. Only then is a live base connected.

Function	Test	Pass condition
E	Turn off the laptop's Wi-Fi for 10 minutes, then back on	Dashboard shows "last synced"; queued actions run after reconnect
E	Log in as a Viewer assigned to one company	Sees only that company; Approve and Create buttons hidden; API returns 403
E	Edit a document in 1C	Change appears in the app within 5 minutes
B	Compare cash, receivables, and OCB with 1C's own reports for one month	Every figure matches to the sum
B	Ask AI: "Why is Kacca negative?"	Answer cites the days and documents behind it
D	Plant one error for each of the 12 rules in a test copy	Each produces exactly one finding with an explanation
C	Approve a VAT-RATE fix, then undo it	1C shows the change, then the original values; both logged
C	Try a fix dated in a closed period	Refused with 409; nothing changes in 1C
A	Create, post, and "send" 3 invoices, one by Excel upload	Numbers, totals, and VAT match in 1C; statuses correct
A	Submit an invoice with an invalid INN	Blocked before it reaches 1C
AI	Ask a question about all 4 companies from Claude.ai via the MCP connector	Correct answer; a Viewer token cannot reach write tools
Security	Call the extension from another PC on the network	Connection refused